**Name: Gaurang Vaghela**
**Rollno: BEAD-22561**
**CI Practical 5**
**Problem Statement:** Optimization of genetic algorithm parameter in hybrid genetic algorithm neural network modelling: Application to spray drying of coconut milk.

**Code:**

```
import numpy as np
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
import random

# -------------------------------
# Generate sample data
X = np.random.rand(100, 5)
y = np.sum(X, axis=1) + np.random.rand(100) * 0.1  # realistic target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# -------------------------------
# Fitness function
def fitness(params):
    pop_size, crossover_rate, mutation_rate = params

    # Neural Network
    model = MLPRegressor(hidden_layer_sizes=(50,),
                         max_iter=300,
                         random_state=42)

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)

    return mse  # lower is better

# -------------------------------
# Genetic Algorithm

def create_individual():
```

```
    return [
        random.randint(50, 100),        # population size
        random.uniform(0.6, 0.9),       # crossover rate
        random.uniform(0.01, 0.1)       # mutation rate
    ]

def mutate(ind):
    if random.random() < 0.3:
        ind[0] = random.randint(50, 100)
    if random.random() < 0.3:
        ind[1] = random.uniform(0.6, 0.9)
    if random.random() < 0.3:
        ind[2] = random.uniform(0.01, 0.1)
    return ind

def crossover(p1, p2):
    return [
        random.choice([p1[0], p2[0]]),
        random.choice([p1[1], p2[1]]),
        random.choice([p1[2], p2[2]])
    ]

# ------------------------------
# GA Main Loop

population = [create_individual() for _ in range(10)]

for generation in range(10):
    scores = [(fitness(ind), ind) for ind in population]
    scores.sort(key=lambda x: x[0])  # minimize error

    print(f"Generation {generation} Best MSE:", scores[0][0])

    # selection (top 50%)
    selected = [ind for (_, ind) in scores[:5]]

    new_population = selected.copy()

    while len(new_population) < 10:
        p1, p2 = random.sample(selected, 2)
        child = crossover(p1, p2)
        child = mutate(child)
        new_population.append(child)
```

```
    population = new_population

best = min(population, key=lambda x: fitness(x))

print("\nBest Parameters Found:")
print("Population Size:", best[0])
print("Crossover Rate:", best[1])
print("Mutation Rate:", best[2])
```

**OUTPUT:**

```
vboxuser@Ubuntu-24: ~/Desktop/CI5
vboxuser@Ubuntu-24:~/Desktop/CI5$ python3 CI5.py
Generation 0 Best MSE: 0.04239393553329943
Generation 1 Best MSE: 0.04239393553329943
Generation 2 Best MSE: 0.04239393553329943
Generation 3 Best MSE: 0.04239393553329943
Generation 4 Best MSE: 0.04239393553329943
Generation 5 Best MSE: 0.04239393553329943
Generation 6 Best MSE: 0.04239393553329943
Generation 7 Best MSE: 0.04239393553329943
Generation 8 Best MSE: 0.04239393553329943
Generation 9 Best MSE: 0.04239393553329943

Best Parameters Found:
Population Size: 91
Crossover Rate: 0.6829747661790283
Mutation Rate: 0.024941023508679494
vboxuser@Ubuntu-24:~/Desktop/CI5$
```